19

Factoring integers

In this chapter, we present several of the algorithms listed in Table 19.1 to factor an integer N of length n into its prime divisors. The running time of Lenstra's algorithm actually depends not on n, but mainly on the size of the second largest prime factor of N. Some of the timing analyses are only heuristic, not rigorously proven. We note that the input size is $n \approx \log_2 N/64$ words.

method	year	time
trial division	$-\infty$	$O^{\sim}(2^{n/2})$
Pollard's $p-1$ method	1974	$O^{\sim}(2^{n/4})$
Pollard's ρ method	1975	$O^{\sim}(2^{n/4})$
Pollard's and Strassen's method	1976	$O^{\sim}(2^{n/4})$
Morrison's and Brillhart's continued fractions	1975	$\exp(O^{\sim}(n^{1/2}))$
Dixon's random squares	1981	$\exp(O^{\sim}(n^{1/2}))$
Lenstra's elliptic curves	1987	$\exp(O^{\sim}(n^{1/2}))$
number field sieve	1990	$\exp(O^{\sim}(n^{1/3}))$

TABLE 19.1: Some algorithms to factor an integer of length n.

The reader will become convinced that a fair amount of mathematical ingenuity has been spent on this problem, and that modern methods can attack surprisingly large numbers. But in comparison to the striking success in factoring polynomials (say, of degree 1 000 000 over $\mathbb{F}_2$), the current records, still under 300 digits, are disappointingly small. This is a manifestation of the practical relevance of polynomial time.

In Chapter 20, we will see how this disappointment has been turned around to major progress in another area: the assumed difficulty of factoring is important for the security of some cryptosystems.

19.1. Factorization challenges

A driving force and benchmark for the development of integer factoring algorithms is the **Cunningham project**. Cunningham & Woodall (1925) published a table of

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

factorizations of $b^n \pm 1$ for nonsquare b with $2 \le b \le 12$. The book by Brillhart, Lehmer, Selfridge, Tuckerman & Wagstaff (1988) contains substantial extensions of these tables, and Sam Wagstaff regularly keeps the world up to date with a newsletter on progress and the "most wanted" factorizations.[1]

If it were easy to factor large integers, then cryptosystems like RSA (Section 20.2) would be insecure. RSA Labs, a company marketing these security products, posed the *RSA challenge* of factoring certain integers. An implementation of the number field sieve (see Notes 19.7) factored in 2009 the 232-digit RSA challenge RSA-768 into its two prime factors (Kleinjung, Aoki, Franke, Lenstra, Thomé, Bos, Gaudry, Kruppa, Montgomery, Osvik, te Riele, Timofeev & Zimmermann 2010).

Another challenge for a new algorithm or implementation is to factor one of the Cunningham numbers. The number field sieve gained instant fame when it was used to factor the ninth Fermat number $F_9 = 2^{2^9} + 1$ (Section 18.2).

n	Factorization	Method	Discovery
5	$p_3 \cdot p_7$		Euler (1732/33, 1747/48)
6	$p_6 \cdot p_{14}$	unknown	Landry (1880); see Williams (1993)
7	$p_{17} \cdot p_{22}$	cont. fractions	Morrison & Brillhart (1971)
8	$p_{16} \cdot p_{62}$	Pollard ρ	Brent & Pollard (1981)
9	$p_7 \cdot p_{49} \cdot p_{99}$	number field sieve	Lenstra *et al.* (1990)
10	$p_8 \cdot p_{10} \cdot p_{40} \cdot p_{252}$	elliptic curves	Brent (1995)
11	$p_6 \cdot p'_6 \cdot p_{21} \cdot p_{22} \cdot p_{564}$	elliptic curves	Brent (1988)

TABLE 19.2: Factorization of Fermat numbers F_n.

The elliptic curve method has been successful in finding some of the "most wanted" factorizations of the Cunningham project, in particular for some Fermat numbers. Table 19.2 shows the history of factorizations of Fermat numbers F_n; p_k stands of a prime number with k decimal digits. Richard Brent (1999) reports on two factorizations that he calculated:

$$F_{10} = 45\,592\,577 \cdot 6\,487\,031\,809 \cdot 4\,659\,775\,785\,220\,018\,543\,264\,560\,743\,076\,778\,192\,897 \cdot p_{252},$$

$$F_{11} = 319\,489 \cdot 974\,849 \cdot 167\,988\,556\,341\,760\,475\,137 \cdot 3\,560\,841\,906\,445\,833\,920\,513 \cdot p_{564},$$

He factored the 617-digit number F_{11} in 1988, and F_{10} (with 309 decimal digits) in 1995; the former was easier for his elliptic curve software because its second-largest prime divisor has 22 digits vs. 40 digits for F_{10} and it is this second-largest divisor that determines the running time of the elliptic curve algorithm. Numbers of this size are typically outside the reach of modern factorization software. The next Fermat number factors as

$$F_{12} = 114\,689 \cdot 26\,017\,793 \cdot 63\,766\,529 \cdot 190\,274\,191\,361 \cdot 1\,256\,132\,134\,125\,569 \cdot c,$$

[1] `http://www.cerias.purdue.edu/homes/ssw/cun/index.html`

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

where c is a composite integer with 1187 digits. These factorizations require a careful implementation and a slew of special insights and tricks, devised by the people working in the area. All F_n with $5 \le n \le 30$ are known to be composite. Determining this for F_{24} took as many computer operations as the rendering of the feature-length Pixar-Disney movie *A Bug's Life*.

In Wagstaff's Cunningham list dated 24 March 2011, the top "most wanted" factorization was that of $2^{929} - 1$. Three prime factors have been found, but the remaining cofactor is composite and has 214 decimal digits.

In order to factor an integer N, one will first remove all "small" prime factors, say below 10^6, by taking gcd's with the product of several of these; see also Exercise 19.5. So we may assume N to be odd, and also composite, which can be ascertained by some probabilistic primality test (Chapter 18). Some algorithms assume in addition that N is not a perfect power, so that we first use the algorithm in Section 9.5 to check whether N is an rth power for some integer r, and replace N by its rth root for the largest possible r (Exercise 9.44). Exercise 18.6 describes how the strong primality test 18.5 can be modified so as to recognize prime powers as well. If successful, most algorithms return one nontrivial divisor d of N, and to obtain the complete factorization of N, the whole procedure—including primality and perfect power test—is applied recursively to d and N/d. Figure 19.3 illustrates the whole process. We only discuss probabilistic algorithms, which guarantee the primality of the returned factors with high probability. Some applications require a rigorous proof of their primality, which is provided by the deterministic method of Agrawal, Kayal & Saxena (2004).

19.2. Trial division

The following is the simplest factoring algorithm.

ALGORITHM 19.1 Trial division.
Input: $N \in \mathbb{N}_{\ge 3}$, neither a prime nor a perfect power, and $b \in \mathbb{N}$.
Output: The smallest prime factor of N if it is less than b, and otherwise "failure".

1. **for** $p = 2, 3, \ldots, b$ **do**
2. **if** $p \mid N$ **then return** p
3. **return** "failure"

To find all prime factors, we divide out p as often as possible and continue as illustrated in Figure 19.3. When calling the algorithm again (with some larger value of b in case of failure), we may of course use that the input has no prime factors below p. This procedure will terminate when p is the second largest prime factor of N.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

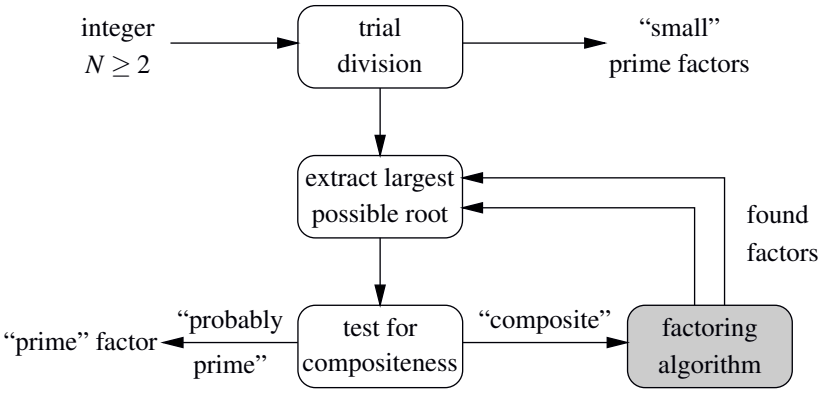


FIGURE 19.3: Factoring an integer into primes.

For $N \in \mathbb{N}$, we let $S_1(N)$ denote the largest prime divisor of N, and $S_2(N)$ the second largest prime divisor of N. Thus $S_2(N) < N^{1/2}$. The number of steps required by trial division is $S_2(N)(\log N)^{O(1)}$. For random integers N,

$$\text{prob}(S_1(N) > N^{0.85}) \approx 0.20, \quad \text{prob}(S_2(N) > N^{0.30}) \approx 0.20.$$

Thus, the number of steps needed for the trial division algorithm is $O^{\sim}(N^{0.30})$ "most of the time".

19.3. Pollard's and Strassen's method

We denote by $a \longmapsto \overline{a}$ the reduction of integers modulo N. Let $1 \leq c \leq \sqrt{N}$, $F = (x+1)(x+2)\dots(x+c) \in \mathbb{Z}[x]$, and $f = \overline{F} \in \mathbb{Z}_N[x]$. Then

$$\overline{c^2!} = \prod_{0 \leq i < c} f(\overline{ic}).$$

The following algorithm uses a "baby step/giant step" strategy.

ALGORITHM 19.2 Pollard's and Strassen's integer factoring algorithm.
Input: $N \in \mathbb{N}_{\geq 3}$, neither a prime nor a perfect power, and $b \in \mathbb{N}$.
Output: The smallest prime factor of N if it is less than b, or otherwise "failure".

1. $c \longleftarrow \lceil b^{1/2} \rceil$
   call Algorithm 10.3 to compute the coefficients of $f = \prod_{1 \leq j \leq c} (x + \overline{j}) \in \mathbb{Z}_N[x]$

2. **call** the fast multipoint evaluation algorithm 10.7 to compute $g_i \in \{0, \dots, N-1\}$ such that $g_i \bmod N = f(\overline{ic})$ for $0 \leq i < c$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

3. **if** $\gcd(g_i,N) = 1$ for $0 \le i < c$ **then return** "failure"
   $k \longleftarrow \min\{0 \le i < c : \gcd(g_i,N) > 1\}$

4. **return** $\min\{kc+1 \le d \le kc+c : d \mid N\}$

We recall the multiplication time M (see inside back cover).

THEOREM 19.3.
Algorithm 19.2 works correctly and uses $O(\mathsf{M}(b^{1/2})\mathsf{M}(\log N)(\log b + \operatorname{loglog} N))$ *word operations and space for* $O(b^{1/2}\log N)$ *words.*

PROOF. For $0 \le i < c$, a prime divisor p of N divides $F(ic)$, and hence also $\gcd(g_i,N) = \gcd(F(ic) \operatorname{rem} N, N)$, if and only if p divides some number in the interval $\{ic+1,\ldots,ic+c\}$, and the correctness follows.

By Lemma 10.4 and Corollary 10.8, the cost for steps 1 and 2 is $O(\mathsf{M}(c)\log c)$ additions and multiplications in $\mathbb{Z}_N$. Step 3 takes $O(c\mathsf{M}(\log N)\operatorname{loglog} N)$ word operations, as noted in the end of Section 11.1, and step 4 takes $O(c\mathsf{M}(\log N))$ word operations, by Theorem 9.8. The cost for one addition or multiplication in $\mathbb{Z}_N$ is $O(\mathsf{M}(\log N))$, by Corollary 9.9, and we get a total cost of $O(\mathsf{M}(b^{1/2})\mathsf{M}(\log N) \cdot (\log b + \operatorname{loglog} N))$. We have to store $O(b^{1/2})$ integers of length $O(\log N)$. □

If we run Algorithm 19.2 for $b = 2^i$ and $i = 1,2,\ldots$, then we have factored N completely as soon as $b > S_2(N)$. This leads to the following result.

COROLLARY 19.4.
Using Algorithm 19.2, we can completely factor N *with*

$$O\left(\mathsf{M}(S_2(N)^{1/2})\mathsf{M}(\log N)\log N\right) \text{ or } O^{\sim}(N^{1/4})$$

word operations and space $O^{\sim}(N^{1/4})$.

19.4. Pollard's rho method

This probabilistic method is by Pollard (1975). Its time bounds are "heuristic" and have not been rigorously established. The idea is as follows. We choose some function $f : \mathbb{Z}_N \longrightarrow \mathbb{Z}_N$ and a starting value $x_0 \in \mathbb{Z}_N$, and recursively define $x_i \in \mathbb{Z}_N$ by $x_i = f(x_{i-1})$ for all $i > 0$. We hope that $x_0, x_1, x_2, \ldots$ behave like a sequence of independent random elements in $\mathbb{Z}_N$. If p is an (unknown) prime divisor of N, then we will have a collision modulo p if there are two integers t, l with $l > 0$ and $x_t \equiv x_{t+l} \bmod p$. If N is not a prime power and q is a different prime divisor of N, and the x_i's are random residues modulo N, then $x_i \bmod p$ and $x_i \bmod q$ are

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

independent random variables, by the Chinese Remainder Theorem. Thus it is very likely that $x_t \not\equiv x_{t+l} \bmod q$, and $\gcd(x_t - x_{t+l}, N)$ is a nontrivial factor of N.

The first question now is: how "small" can we expect t and l to be? We certainly have $t + l \leq p$, and the following analysis shows that the expected value is only $O(\sqrt{p})$ for a *random* sequence $(x_i)_{i\in\mathbb{N}}$. The problem is known as the **birthday problem:** Assuming peoples' birthdays occur randomly, how many people do we need to get together before we have probability at least $1/2$ for at least two people to have the same birthday? Surprising answer: only 23 are sufficient. In fact, with 23 or more people at a party, the probability of two coinciding birthdays is at least 50.7%.

THEOREM 19.5 *Birthday problem.*
We consider random choices, with replacement, among p labeled balls. The expected number of choices until a collision occurs is $O(\sqrt{p})$.

PROOF. Let s be the number of choices until a collision occurs, that is, two identical balls are chosen. This is a random variable. For $j \geq 2$, we have

$$\begin{aligned}\operatorname{prob}(s \geq j) &= \frac{1}{p^{j-1}} \prod_{1\leq i<j} (p-(i-1)) \;=\; \prod_{1\leq i<j} \left(1 - \frac{i-1}{p}\right) \\ &\leq \prod_{1\leq i<j} e^{-(i-1)/p} = e^{-(j-1)(j-2)/2p} \leq e^{-(j-2)^2/2p},\end{aligned}$$

where we have used $1 - x \leq e^{-x}$. It follows that

$$\begin{aligned}\mathcal{E}(s) &= \sum_{j\geq 1} \operatorname{prob}(s \geq j) \leq 1 + \sum_{j\geq 0} e^{-j^2/2p} \leq 2 + \int_0^\infty e^{-x^2/2p} dx \\ &\leq 2 + \sqrt{2p} \int_0^\infty e^{-x^2} dx = 2 + \sqrt{\frac{p\pi}{2}}\end{aligned}$$

since $\int_0^\infty e^{-x^2} dx = \sqrt{\pi}/2$, by Exercise 19.6. □

Floyd's cycle detection trick. Given an integer $x_0 \in \{0, \ldots, p-1\}$ and a function $f: \{0, \ldots, p-1\} \longrightarrow \{0, \ldots, p-1\}$, we examine the sequence $x_0, x_1, \ldots$ defined by $x_{i+1} = f(x_i)$ for $i \geq 0$. This is an infinite sequence from a finite set, so that at some point the values repeat. This results in a cycle of some length $l > 0$ such that $x_i = x_{i+l}$ for all $i \geq t$, for some $t \in \mathbb{N}$. We may assume that l and t are minimal with that property. In Figure 19.4, we see an example with $t = 3$ and $l = 7$.

An obvious method to find $i \neq j$ such that $x_i = x_j$ is to write down the sequence until a value repeats itself, but this requires space $O(t + l)$. The following algorithm uses only constant space. The idea of Floyd's 1-step/2-step cycle detection method

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

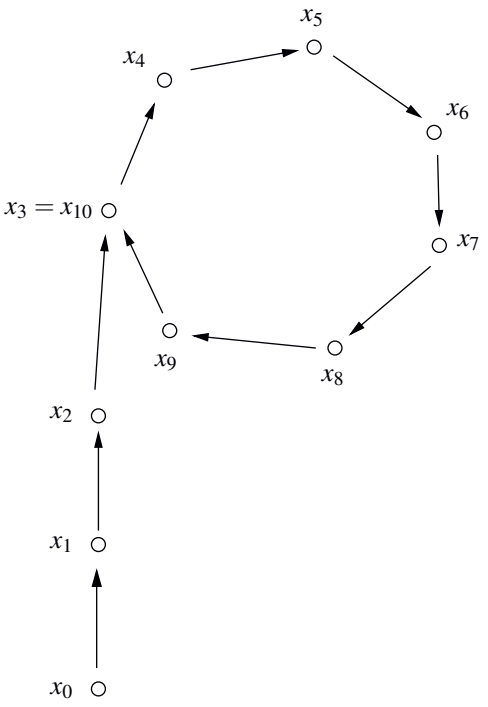


FIGURE 19.4: Pollard's ρ method.

is to use a second sequence $(y_i)_{i\in\mathbb{N}}$ that iterates f with double speed, so that $y_i = x_{2i}$ for all i, and to store only the current values of x_i and y_i. Intuitively, it should be clear that the "faster" sequence "overtakes" the slower one for some i, and then we have $x_{2i} = y_i = x_i$.

ALGORITHM 19.6 Floyd's cycle detection trick.
Input: $x_0 \in \{0,\dots,p-1\}$ and $f\colon\{0,\dots,p-1\} \longrightarrow \{0,\dots,p-1\}$, as above.
Output: An index $i > 0$ such that $x_i = x_{2i}$.

1. $y_0 \longleftarrow x_0, \quad i \longleftarrow 0$
2. **repeat** $i \longleftarrow i+1, \quad x_i \longleftarrow f(x_{i-1}), \quad y_i \longleftarrow f(f(y_{i-1}))$
   until $x_i = y_i$
3. **return** i

The following lemma says that the number of steps until the first collision $x_i = y_i$ in Floyd's method occurs is at most the number of steps until the first collision $x_i = x_j$ with $i < j$ happens.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

LEMMA 19.7. *With t and l as above, Algorithm 19.6 halts after at most $t+l$ steps.*

PROOF. Since $y_i = x_{2i}$ for all i, we have $x_i = y_i$ if and only if $i \geq t$ and $l \mid (2i-i) = i$, and the smallest positive such index is $i = t + (-t \text{ rem } l) < t+l$ if $t > 0$, and $i = l$ if $t = 0$. □

We now describe Pollard's ρ method for factoring N. It generates a sequence $x_0, x_1, \ldots \in \{0 \ldots N-1\}$. We pick x_0 at random and define $x_{i+1} \in \{0, \ldots, N-1\}$ by $x_{i+1} = f(x_i) = x_i^2 + 1 \text{ rem } N$. The choice of this iteration function is black magic, but linear functions do not work, and higher degree polynomials are more costly to evaluate, and one cannot prove more about them than about x^2+1.

Let p be the smallest prime dividing N. Then we have $x_{i+1} \equiv x_i^2 + 1 \bmod p$ for $i \geq 0$. The birthday problem and heuristic reasoning which says that the x_i's "look" random imply that we can expect a collision mod p after $O(\sqrt{p})$ steps. This duplicate can be detected by using Floyd's cycle detection trick.

ALGORITHM 19.8 Pollard's ρ method.
Input: $N \in \mathbb{N}_{\geq 3}$, neither a prime nor a perfect power.
Output: Either a proper divisor of N, or "failure".

1. pick $x_0 \in \{0, \ldots, N-1\}$ at random, $\quad y_0 \longleftarrow x_0, \quad i \longleftarrow 0$
2. **repeat**
3. $\quad i \longleftarrow i+1, \quad x_i \longleftarrow x_{i-1}^2 + 1 \text{ rem } N, \quad y_i \longleftarrow (y_{i-1}^2+1)^2 + 1 \text{ rem } N$
4. $\quad g \longleftarrow \gcd(x_i - y_i, N)$
   if $1 < g < N$ **then return** g
   else if $g = N$ **then return** "failure"

THEOREM 19.9.
Let $N \in \mathbb{N}$ be composite, p its smallest prime factor, and $f(x) = x^2+1$. Under the assumption that the sequence $(f^i(x_0))_{i \in \mathbb{N}}$ behaves modulo p like a random sequence, the expected running time of Pollard's algorithm for finding the smallest prime factor p of N is $O(\sqrt{p}\,\mathsf{M}(\log N)\log\log N)$. By applying the algorithm recursively, N can be completely factored in expected time $S_2(N)^{1/2} O^{\sim}(\log^2 N)$, or $O^{\sim}(N^{1/4})$.

EXAMPLE 19.10. We want to factor $N = 82\,123$. Starting with $x_0 = 631$, we find the following sequence:

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

i	$x_i \bmod N$	$x_i \bmod 41$
0	631	16
1	69 670	11
2	28 986	40
3	69 907	2
4	13 166	5
5	64 027	26

i	$x_i \bmod N$	$x_i \bmod 41$
6	40 816	21
7	80 802	32
8	20 459	0
9	71 874	1
10	6685	2

The iteration modulo 41 is illustrated in Figure 19.4; the algorithm's name derives from the similarity to the Greek letter ρ. It leads to the factor $\gcd(x_3 - x_{10}, N) = 41$.

When we execute the algorithm, only the values modulo N are known; the values modulo 41 are included for our understanding. The algorithm calculates in tandem x_i and $y_i = x_{2i}$ and performs the gcd test each time. We have $t = 3$, $l = 7$, and $t + (-t \text{ rem } l) = 7$, and in fact, after seven 2-steps the algorithm catches up with the 1-steps:

i	$x_i \bmod N$	$x_i \bmod 41$	$y_i \bmod N$	$y_i \bmod 41$	$\gcd(x_i - y_i, N)$
0	631	16	631	16	1
1	69 670	11	28 986	40	1
2	28 986	40	13 166	5	1
3	69 907	2	40 816	21	1
4	13 166	5	20 459	0	1
5	64 027	26	6685	2	1
6	40 816	21	75 835	26	1
7	80 802	32	17 539	32	41

The factorization $N = 41 \cdot 2003$ is found as $\gcd(x_7 - y_7, N) = 41$. Of course the sequence also repeats modulo 2003, and in fact $x_{38} \equiv 4430 \equiv x_{143} \bmod N$. If this repetition occurred simultaneously with the one modulo 41, we would not find a factor: an "unlucky" x_0. $\diamond$

19.5. Dixon's random squares method

In this section, we present Dixon's (1981) method of using random squares for factoring an odd integer N that is not a prime power. It was the first method proven to be faster than $\exp(\varepsilon \cdot \log N)$ for any $\varepsilon > 0$. Our explanation of Dixon's method proceeds in four stages. We first describe the idea of the algorithm, including an example, and then state the algorithm, with an unspecified parameter B. The crux then is to show that sufficiently many random numbers computed in the algorithm are smooth, that is, they have only prime factors below B. From this, we can determine the best value for B and complete the analysis.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

We begin by observing that the equations

$$N = s^2 - t^2 = (s+t)(s-t),$$
$$N = a \cdot b = \left(\frac{a+b}{2}\right)^2 - \left(\frac{a-b}{2}\right)^2$$

describe a bijection between factorizations of N and representations of N as a difference of two squares. This immediately suggests a crude factorization algorithm: for $t = \lceil\sqrt{N}\rceil, \lceil\sqrt{N}\rceil + 1, \ldots$, check whether $t^2 - N$ is a perfect square. If we find such a square, then we can factor N. This algorithm will work well if $N = ab$ with $|a-b|$ small, since then the running time is dependent on $|a-b|$. This was already clear to Fermat; he took $N = 2\,027\,651\,281$, so that $\sqrt{N} \approx 45\,029$, and found

$$N = 45\,041^2 - 1020^2 = 46\,061 \cdot 44\,021.$$

A modification is to choose some integer $k \ll N$, to let $t = \lceil\sqrt{kN}\rceil, \lceil\sqrt{kN}\rceil + 1, \ldots$, and then to check if $t^2 - kN$ is a perfect square. If $t^2 - kN = s^2$, then $\gcd(s+t, N)$ is a factor of N, hopefully nontrivial, so that $s \not\equiv \pm t \bmod N$.

Actually, finding a relation $s^2 \equiv t^2 \bmod N$ in this way is unlikely for large N. The basic idea of the random squares method is illustrated in the following example.

EXAMPLE 19.11. Let $N = 2183$. Suppose that we have found the system of congruences

$$453^2 \equiv 7 \bmod N, \quad 1014^2 \equiv 3 \bmod N, \quad 209^2 \equiv 21 \bmod N.$$

Then we obtain $(453 \cdot 1\,014 \cdot 209)^2 \equiv 21^2 \bmod N$, or $687^2 \equiv 21^2 \bmod N$. This yields the factors $37 = \gcd(687 - 21, N)$ and $59 = \gcd(687 + 21, N)$; in fact $N = 37 \cdot 59$ is the prime factorization of N. $\diamond$

The above example suggests a systematic approach: choose b at random and hope that b^2 rem N is a product of small primes. When enough have been found, we will get a congruence $s^2 \equiv t^2 \bmod N$. We now describe this approach in detail.

We take as our **factor base** the prime numbers $p_1, p_2, \ldots, p_h$ up to B, for some parameter $B \in \mathbb{R}_{>0}$. A number b is a ***B*-number** if the integer b^2 rem N (the remainder of b^2 on division by N) is a product of these primes. Thus, in Example 19.11, the integers 453, 1014, and 209 are B-numbers, for any $B \geq 7$ and $N = 2183$. A variant of the algorithm allows $p_0 = -1$ in the factor base and considers the least absolute residue a of $b \in \mathbb{Z}$, defined by $b \equiv a \bmod N$ and $-N/2 < a \leq N/2$; we do not pursue this here.

For every B-number b, we write $b^2 \equiv p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_h^{\alpha_h} \bmod N$, with $\alpha_1, \ldots, \alpha_h \in \mathbb{N}$, and associate with b the binary exponent vector

$$\varepsilon = (\alpha_1 \bmod 2, \alpha_2 \bmod 2, \ldots, \alpha_h \bmod 2) \in \mathbb{F}_2^h. \tag{1}$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

Now suppose that we have $b_1, b_2, \ldots, b_l$ such that $\varepsilon_1 + \varepsilon_2 + \cdots + \varepsilon_l = 0$ in $\mathbb{F}_2^h$. Writing $b_i^2 \equiv \prod_{1 \le j \le h} p_j^{\alpha_{ij}} \bmod N$, we see that

$$\left(\prod_{1 \le i \le l} b_i \right)^2 \equiv \prod_{1 \le j \le h} p_j^{\sum_{1 \le i \le l} \alpha_{ij}} = \prod_{1 \le j \le h} p_j^{2\gamma_j} = \left(\prod_{1 \le j \le h} p_j^{\gamma_j} \right)^2 \bmod N,$$

where $\gamma_j = \frac{1}{2} \sum_{1 \le i \le l} \alpha_{ij} \in \mathbb{N}$ for all j. This gives the desired congruence $s^2 \equiv t^2 \bmod N$, where

$$s = \prod_{1 \le i \le l} b_i, \quad t = \prod_{1 \le j \le h} p_j^{\gamma_j}. \tag{2}$$

Note that we have to generate no more than $h+1$ B-numbers, so that we may always take $l \le h+1$, since any set of $h+1$ vectors in $\mathbb{F}_2^h$ is linearly dependent over $\mathbb{F}_2$.

Having obtained $s^2 \equiv t^2 \bmod N$, we hope that $s \not\equiv \pm t \bmod N$. If N is not a prime power and has $r \ge 2$ distinct prime factors, then the Chinese Remainder Theorem 5.3 implies that every square in $\mathbb{Z}_N^\times$ has exactly 2^r square roots in $\mathbb{Z}_N$. Therefore, if s is a random square root of t^2 we have

$$\text{prob}\{s \equiv \pm t \bmod N\} = \frac{2}{2^r} \le \frac{1}{2}.$$

In Example 19.11, with $B = \{2,3,5,7\}$, we have $\varepsilon_1 = (0,0,0,1)$, $\varepsilon_2 = (0,1,0,0)$, $\varepsilon_3 = (0,1,0,1)$ and $\varepsilon_1 + \varepsilon_2 + \varepsilon_3 = 0$ in $\mathbb{F}_2^4$. Furthermore $\gamma_1 = \gamma_3 = 0$, $\gamma_2 = \gamma_4 = 1$, $s = 453 \cdot 1014 \cdot 209$, and $t = 2^0 \cdot 3^1 \cdot 5^0 \cdot 7^1$.

Here is the resulting algorithm.

ALGORITHM 19.12 Dixon's random squares method.
Input: An odd integer $N \ge 3$, neither a prime nor a perfect power, and $B \in \mathbb{R}_{>0}$.
Output: Either a proper divisor of N, or "failure".

1. compute all primes $p_1, p_2, \ldots, p_h$ up to B
   if p_i divides N for some $i \in \{1, \ldots, h\}$ **then return** p_i

2. $A \longleftarrow \emptyset$ { initialize the set of B-numbers }
   repeat

3. choose a uniform random number $b \in \{2, \ldots, N-2\}$
   $g \longleftarrow \gcd(b, N)$, **if** $g > 1$ **then return** g

4. $a \longleftarrow b^2 \text{ rem } N$
   { factor a over $\{p_1, \ldots, p_h\}$ }
   for $i = 1, \ldots, h$ **do**

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

5. { determine multiplicity of p_i in a }
   $\alpha_i \longleftarrow 0$
   while p_i divides a **do** $a \longleftarrow \dfrac{a}{p_i}, \quad \alpha_i \longleftarrow \alpha_i + 1$

6. **if** $a = 1$ **then** $\alpha \longleftarrow (\alpha_1, \ldots, \alpha_h), \quad A \longleftarrow A \cup \{(b, \alpha)\}$

7. **until** $\#A = h + 1$

8. find distinct pairs $(b_1, \alpha^{(1)}), \ldots, (b_l, \alpha^{(l)}) \in A$ with $\alpha^{(1)} + \cdots + \alpha^{(l)} \equiv 0 \bmod 2$ in $\mathbb{F}_2^h$, for some $l \geq 1$, by solving an $(h+1) \times h$ system of linear equations over $\mathbb{F}_2$

9. $(\gamma_1, \ldots, \gamma_h) \longleftarrow \dfrac{1}{2}(\alpha^{(1)} + \cdots + \alpha^{(l)})$
   $s \longleftarrow \displaystyle\prod_{1 \leq i \leq l} b_i, \quad t \longleftarrow \prod_{1 \leq j \leq h} p_j^{\gamma_j}, \quad g \longleftarrow \gcd(s + t, N)$
   if $g < N$ **then return** g **else return** "failure"

We let $n = \log N$. Using the sieve of Eratosthenes, the cost for setting up the factor base in step 1 is $O(h \log^2 h \log\log h)$ word operations, by Theorem 18.10, and the cost for checking divisibility is $O(h \cdot \mathsf{M}(n))$. The cost for one iteration of the loop 2 is $O(\mathsf{M}(n) \log n)$ word operations for the gcd, $O(\mathsf{M}(n))$ word operations to compute b^2 rem N, and $O((h+n)\mathsf{M}(n))$ operations for trial division by all primes $p_1, \ldots, p_h$ to check smoothness. (The check can actually be performed faster with a modification of the Pollard and Strassen algorithm 19.2.) If k is the number of iterations of the loop 2, then the total cost of the loop is $O(k(h+n)\mathsf{M}(n))$ word operations. The cost of solving the system of linear equations over $\mathbb{F}_2$ in step 8 is $O(h^3)$ word operations. The cost of all other steps is dominated by these estimates, and we obtain a total cost of

$$O(h^3 + k(h+n)\mathsf{M}(n)) \tag{3}$$

word operations.

In practice, -1 is included in B and the numbers b are chosen randomly in the vicinity of $\sqrt{N}$, because the least absolute residue of b^2 modulo N is then only about $O(\sqrt{N})$ and more likely to have all prime factors less than B than an arbitrary number up to N. For our arguments below, however, we need the assumption that the b's are uniform random numbers between 1 and $N-1$.

Our goal now is to estimate the expected number k of iterations, and to determine the right choice for B, given just N. For given $x, y \in \mathbb{R}_{>2}$, we let

$$\begin{aligned} \Psi(x,y) &= \{a \in \mathbb{N} : 1 \leq a \leq x, \forall p \text{ prime } p \mid a \Longrightarrow p \leq y\}, \\ \psi(x,y) &= \#\Psi(x,y). \end{aligned} \tag{4}$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

The numbers in $\Psi(x,y)$, all of whose prime factors are not greater than y, are called **y-smooth**. We have

$$b \text{ is a } B\text{-number} \iff b^2 \text{ rem } N \in \Psi(N,B).$$

Clearly the crux of the problem is to choose B wisely: if B is too small, then B-numbers are rare and take a long time to find, and if B is too large, then it takes a long time to test a prospective B-number, and the linear system is large.

As a warmup exercise, we estimate roughly the probability that a random integer $a \in \{1,\dots,x\}$ is y-smooth, with $y = B$. We put $u = \ln(x)/\ln(y)$, so that $y = x^{1/u}$, and $v = \lfloor u \rfloor$, and let $(a_1,a_2,\dots,a_v) \in \{p_1,\dots,p_h\}^v$ and $a = a_1a_2\cdots a_v$. (ln is the natural logarithm.) Then $a \le B^v \le y^u = x$, hence $a \in \Psi(x,y)$. Each a comes from at most $v!$ vectors in $\{p_1,\dots,p_h\}^v$, and hence we have the approximate inequalities

$$\psi(x,y) \ge \frac{h^v}{v!} \ge \left(\frac{h}{v}\right)^v \gtrsim \left(\frac{y}{\ln y}\right)^v \cdot v^{-v} \approx \left(\frac{y}{\ln y}\right)^u \cdot u^{-u} = x(u\ln y)^{-u},$$

by the prime number theorem 18.7 which says that $h \approx y/\ln y$. So for a random positive integer $a \le x$, we have

$$\text{prob}\{a \text{ is } y\text{-smooth}\} = \frac{\psi(x,y)}{x} \gtrsim (u\ln y)^{-u}.$$

We state without proof that for reasonably small u, the true order of this probability, called **Dickman's ρ-function**, is u^{-u} for large enough y. Although we will not use this fact, it is comforting to know that our coarse estimate is not too far off.

THEOREM 19.13.
Let $u\colon \mathbb{N} \longrightarrow \mathbb{R}_{>1}$ be an increasing function with $u(x) \in O(\log x/\operatorname{loglog} x)$. Then the probability that a random integer in $\{1,\dots,\lfloor x \rfloor\}$ is $x^{1/u}$-smooth satisfies

$$\frac{\psi(x,x^{1/u})}{x} = u^{-u(1+o(1))}.$$

Here, $o(1)$ is shorthand for a function which tends to zero as u approaches infinity. The above estimates apply to random values a. We now prove a similar result about b^2 rem N for random values b. Then the expected number of trials necessary to find a single B-number is at most $(u\ln y)^u$ (or, in fact, at most u^u). A different argument is in Exercise 19.10.

LEMMA 19.14. *Suppose that N is not divisible by any of $p_1,\dots,p_h$, and $r \in \mathbb{N}$ with $p_h^{2r} \le N$. Then*

$$\#\{b \in \mathbb{N}\colon 1 \le b < N \text{ and } b^2 \text{ rem } N \in \Psi(N,p_h)\} \ge \frac{h^{2r}}{(2r)!}. \tag{5}$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

PROOF. The idea of the proof is to adapt our warmup strategy to this situation. So we consider power products b of $p_1, \dots, p_h$ with exactly r factors. The square of such a b is smooth, and therefore b is clearly in the set S on the left hand side of (5). But there are not enough of these numbers. It would be sufficient if the product of any two b's, rather than just the squares, was actually a square modulo N. This looks implausible at first. But consider a prime factor q of N. Modulo q, half of the numbers are squares (Lemma 14.7), and we have not only "square · square = square", but also "nonsquare · nonsquare = square" (Exercise 14.8). The same actually holds modulo a power of q. The proof will produce sufficiently many numbers in S by partitioning the set of all b's as above according to their square/nonsquare character modulo all the prime factors of N. Then when we take two b's that are distinct but have the same character, their product will actually be a square modulo all q's, and therefore modulo N. The following proof makes this precise. For a first understanding, the reader may want to assume that $N = q_1 q_2$ is the product of two distinct primes.

To begin the proof, we let $N = q_1^{l_1} \cdots q_t^{l_t}$ be the prime factorization of N. The **quadratic character** $\chi_i = \chi_{q_i^{l_i}}$ on $\mathbb{Z}^\times_{q_i^{l_i}}$ is defined as follows:

$$\chi_i(a \bmod q_i^{l_i}) = \begin{cases} 1 & \text{if } \exists b \in \mathbb{N}\ a \equiv b^2 \bmod q_i^{l_i}, \\ -1 & \text{otherwise.} \end{cases}$$

Relatives of this character have played a role (implicitly) in equal-degree factorization (Section 14.3) and primality testing (Chapter 18), then called the Jacobi symbol. The map χ_i is a group homomorphism. Putting all these characters together, we get

$$\begin{aligned} \chi\colon \mathbb{Z}_N^\times &\longrightarrow \{1,-1\}^t = G, \\ a \bmod N &\longmapsto \Big(\chi_1(a \bmod q_1^{l_1}), \dots, \chi_t(a \bmod q_t^{l_t})\Big). \end{aligned}$$

We let

$$Q = \{a \in \mathbb{N}\colon 1 \le a < N,\ \gcd(a,N) = 1,\ \text{and } \exists b \in \mathbb{N}\ a \equiv b^2 \bmod N\}$$

be the set of (invertible) squares modulo N. Since a number is a square modulo N if and only if it is modulo each $q_i^{l_i}$, by the Chinese Remainder Theorem 5.3, we have

$$a \in Q \iff \chi(a \bmod N) = (1, \dots, 1)$$

for $1 \le a < N$. Furthermore, an invertible square modulo $q_i^{l_i}$ has precisely two square roots, and, again by the Chinese Remainder Theorem, any $a \in Q$ has precisely 2^t square roots modulo N (Exercise 9.40).

We denote by S the set in (5). For $x \in \mathbb{R}$ and $s \in \mathbb{N}$, we let

$$T_s(x) = \{a \in \mathbb{N}\colon a \le x \text{ and } \exists e_1, \dots, e_h \in \mathbb{N}\ \ a = p_1^{e_1} \cdots p_h^{e_h} \text{ and } e_1 + \cdots + e_h = s\}$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

be the set of p_h-smooth integers below x with exactly s (not necessarily distinct) prime factors. By assumption, we have $a \bmod N \in \mathbb{Z}_N^\times$ for all $a \in T_s(x)$. Now we partition $T_r(\sqrt{N})$ into 2^t sets U_g for $g \in G$:

$$U_g = \{a \in T_r(\sqrt{N}) : \chi(a \bmod N) = g\}.$$

We denote by V the image of the multiplication map

$$\mu : \bigcup_{g \in G} (U_g \times U_g) \longrightarrow \mathbb{N}$$

with $\mu(b,c) = bc$ rem N. Since $\chi(bc \bmod N) = (1,\ldots,1)$ for all $b,c \in U_g$ and $g \in G$, we have $V \subseteq Q$. Furthermore, $V \subseteq T_{2r}(N)$, so that $V \subseteq T_{2r}(N) \cap Q$.

Every element in $T_{2r}(N) \cap Q$ has exactly 2^t square roots, and these are all in S, so that $\#S \geq 2^t \cdot \#(T_{2r}(N) \cap Q)$. How many elements $(b,c) \in \bigcup_{g \in G} U_g \times U_g$ are mapped by μ to the same $a \in V$? Since $b,c \leq \sqrt{N}$ and $bc \equiv a \bmod N$, we then actually have $bc = a$. Thus we have to split the $2r$ prime factors of a into two halves to make up b and c, and there are at most $\binom{2r}{r} = (2r)!/(r!)^2$ ways of doing this. Thus

$$\#V \cdot (2r)!/(r!)^2 \geq \#\Big(\bigcup_{g \in G} U_g \times U_g\Big) = \sum_{g \in G} (\#U_g)^2.$$

Putting things together, we have

$$\#S \geq 2^t \cdot \#(T_{2r}(N) \cap Q) \geq 2^t \cdot \#V \geq 2^t \sum_{g \in G} (\#U_g)^2 \frac{(r!)^2}{(2r)!}.$$

The Cauchy-Schwarz inequality (Exercise 16.10) says that for any two vectors $x = (x_1,\ldots,x_n), y = (y_1,\ldots,y_n) \in \mathbb{R}^n$ we have

$$\sum_{1 \leq i \leq n} x_i^2 \cdot \sum_{1 \leq i \leq n} y_i^2 = \|x\|_2^2 \cdot \|y\|_2^2 \geq x \star y^2 = \Big(\sum_{1 \leq i \leq n} x_i y_i\Big)^2.$$

We apply this to $n = 2^t = \#G$, $x_g = 1$ and $y_g = \#U_g$ for $g \in G$, to obtain

$$2^t \cdot \sum_{g \in G} (\#U_g)^2 \geq \Big(\sum_{g \in G} \#U_g\Big)^2 = \Big(\#T_r(\sqrt{N})\Big)^2.$$

Since $p_h^r \leq \sqrt{N}$, an element of $T_r(\sqrt{N})$ corresponds to a choice of exactly r primes up to p_h, with repetitions. Thus

$$\#T_r(\sqrt{N}) = \binom{h+r-1}{r} \geq \frac{h^r}{r!}.$$

Finally, we find

$$\#S \geq \frac{2^t (r!)^2}{(2r)!} \sum_{g \in G} (\#U_g)^2 \geq \frac{(r!)^2}{(2r)!} \Big(\#T_r(\sqrt{N})\Big)^2 \geq \frac{h^{2r}}{(2r)!}. \quad \square$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

Step 1 of Algorithm 19.12 guarantees that N is not divisible by any of the primes $p_1, \dots, p_h$. Then the expected number of trials to find a single B-number is at most

$$\left(\frac{\#\{B\text{-numbers}\}}{N}\right)^{-1} = \frac{N}{\#\{B\text{-numbers}\}} \le \frac{N(2r)!}{h^{2r}}$$

for any r with $p_h^{2r} \le N$, by Lemma 19.14. We now fix $r \in \mathbb{N}$ and let $n = \ln N$ and $B = N^{1/2r}$, so that $\ln B = n/2r$. By the prime number theorem 18.7, we have $h = \pi(B) > B/\ln(B)$ if $B \ge 59$. Using the rough estimate $(2r)! \le (2r)^{2r}$, we find that the expected number of trials is at most

$$\frac{N}{h^{2r}}(2r)! < \frac{N(\ln B)^{2r}}{B^{2r}}(2r)^{2r} = n^{2r}.$$

We need $h+1$ B-numbers, and the expected number k of loop iterations satisfies $k \le n^{2r}(h+1)$. Plugging this into (3) and using $n < h < B$, we obtain a total cost of

$$O(B^3 + B^2 n^{2r} \mathsf{M}(n)) \tag{6}$$

word operations. Ignoring the factor $\mathsf{M}(n)$ and equating the logarithms of the two factors $B^2 = e^{n/r}$ and $n^{2r} = e^{2r\ln n}$ gives $r^2 \approx n/(2\ln n)$, and we set

$$r = \left\lceil \sqrt{\frac{n}{2\ln n}} \right\rceil. \tag{7}$$

Then $B \le e^{\sqrt{n(\ln n)/2}}$, and using

$$L(N) = e^{\sqrt{\ln N \ln\ln N}}, \tag{8}$$

we obtain the following result by substituting (7) in (6).

THEOREM 19.15.
Dixon's random squares method factors an integer N with an expected number of $O^{\sim}(L(N)^{2\sqrt{2}})$ word operations.

Variants of this algorithm are used for factoring large integers, and many practical improvements to it have been made. We only mention two. The first one notes that, since a number below N has at most $\log_2 N$ factors, each exponent vector (1) has at most $\log_2 N$ nonzero entries, the matrix of the linear system in step 8 is sparse, and we can use a variant of Wiedemann's algorithm (Section 12.4) to solve it in $O^{\sim}(h^2)$ steps. In fact, Wiedemann (1986) invented his algorithm specifically for this approach. However, this does not decrease the cost estimate of Theorem 19.15.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

The second important practical improvement, due to Pomerance (1982, 1985), is to use a sieving method—reminiscent of Eratosthenes' sieve to generate prime numbers—which does not test individual numbers b as above, but rather generates whole series of B-numbers. The amortized cost for a single smoothness test reduces to about $4\log_2\log_2 B$ additions of numbers up to $\log_2 B$. This is called the **quadratic sieve**.

The space requirement for these algorithms is $h\log_2 h$ bits to store the factor base, and roughly the same for the linear algebra, since the matrix has h sparse rows, and Wiedemann's algorithm can be executed in the same space.

19.6. Pollard's $p-1$ method

This algorithm is a useful introduction to the elliptic curve method in the next section. We wish to factor the integer N and assume that N has a prime factor p such that $p-1$ is smooth. Specifically, we assume that all prime powers l^e dividing $p-1$ are such that $l^e \le B$ for some suitably chosen parameter B. The algorithm is as follows.

ALGORITHM 19.16 Pollard's $p-1$ method.
Input: Positive integers $N \ge 3$ and B.
Output: Either a proper divisor of N, or "failure".

1. $k \longleftarrow \operatorname{lcm}\{i : 2 \le i \le B\}$
2. choose $a \in \{2, .., N-2\}$ uniformly at random
3. $b \longleftarrow a^k \bmod N, \quad d \longleftarrow \gcd(b-1, N)$
4. **if** $1 < d < N$ **then return** d **else return** "failure"

We hope that d is a nontrivial divisor of N. Under the above assumption it is certainly true that $a^k \equiv 1 \bmod p$ since $p-1 \mid k$. This guarantees that $d > 1$. In order to have $d < N$ it is sufficient that N have another prime factor q such that $a^k \not\equiv 1 \bmod q$.

Another way to view the above method is as follows. We begin by choosing a group G (in this case $G = \mathbb{Z}_N^\times$) and we hope that the size of the group "$G \bmod p$" is smooth. Then we choose k as a multiple of the order of $G \bmod p$ and $a \in G$ at random, so that a^k is the unit element modulo p, and hope that is not the unit element modulo q. In the next section, $G \bmod p$ will be an elliptic curve over $\mathbb{F}_p$.

19.7. Lenstra's elliptic curve method

In this section, we present Lenstra's (1987) method for factoring integers using elliptic curves. Its running time is expressed by the same function L as for the

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

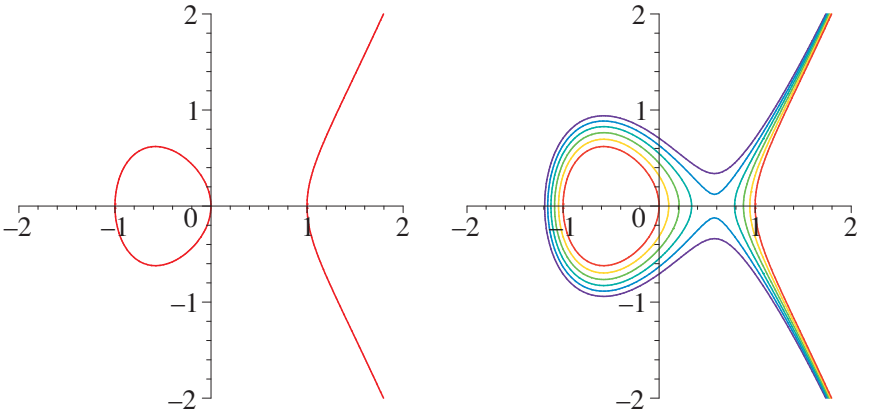


FIGURE 19.5: The elliptic curve $y^2 = x^3 - x$ over the real numbers (left diagram), and the elliptic curves $y^2 = x^3 - x + b$ for $b = 0, 1/10, 2/10, 3/10, 4/10, 5/10$.

random squares method, but instead of $L(N)$ we have $L(p)$, where p is the second largest prime factor of N. Thus it is faster than Dixon's method when $N = pq$ is the product of two primes of substantially different sizes, say 50 and 100 digits. In Section 19.1, we have highlighted some of the successes of this method.

Elliptic Curves. The basic approach in the elliptic curve method to factor N is as follows. One prescribes a certain sequence of computations modulo N. A division by $w \in \mathbb{Z}$ in this sequence can only be executed if $\gcd(w,N) = 1$. Thus at each division step, we either continue the computation or we are lucky—$\gcd(w,N)$ is not trivial and we have found a divisor of N. This is sometimes called the **pretend field technique**. Calculating a multiple of a random point on a random elliptic curve leads to such a sequence of computations. What makes this work is that we will be lucky with reasonably large probability.

The elliptic curve factoring method corresponds to choosing randomly a group G from a set of elliptic curve groups. Lenstra (1987) showed that with large enough probability at least one curve will have smooth order.

We start by defining elliptic curves and stating some of their properties. They inhabit the realm of *algebraic geometry*, one of the richest and deepest areas of mathematics. In this text, we cannot but scratch the surface of this beautiful theory. We have to rely on several results whose proof is beyond the scope of this text.

DEFINITION 19.17. *Let F be a field of characteristic different from 2 and 3, and $x^3 + ax + b \in F[x]$ squarefree. Then*

$$E = \{(u,v) \in F^2 : v^2 = u^3 + au + b\} \cup \{\mathcal{O}\} \subseteq F^2 \cup \{\mathcal{O}\}$$

is an ***elliptic curve*** *over F. Here $\mathcal{O}$ denotes the "point at infinity" on E.*

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

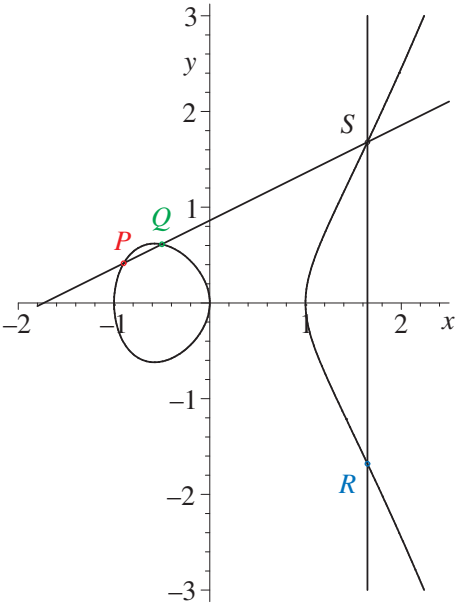


FIGURE 19.6: Adding two points P with $x=-0.9$ (red) and Q with $x=-0.5$ (green) on the elliptic curve $y^2=x^3-x$. The point $R=P+Q$ (blue) is the negative of the intersection point S (black) of the two lines with the curve.

There are other—equivalent—ways of defining and presenting elliptic curves. The above is called the **Weierstraß equation** for E, and a and b are its **Weierstraß coefficients**. The polynomial x^3+ax+b is squarefree if and only if $4a^3+27b^2\neq 0$ (Exercise 19.14).

EXAMPLE 19.18. $x^3-x=x(x-1)(x+1)$ is squarefree if $\operatorname{char} F\neq 2$. The corresponding elliptic curve, together with other examples of elliptic curves, is drawn in Figure 19.5 for $F=\mathbb{R}$. $\diamond$

The reader may imagine that $\mathcal{O}$ lies beyond the horizon in the direction of the y-axis (up *and* down), and that any two vertical lines "intersect" at $\mathcal{O}$. Projective geometry provides a rigorous framework for these notions.

An elliptic curve E is **nonsingular** (or smooth) in the geometric sense, as follows. Let $f=y^2-(x^3+ax+b)\in F[x,y]$, so that $E=\{f=0\}\cup\{\mathcal{O}\}$. For $(u,v)\in E\setminus\{\mathcal{O}\}$, we have

$$\left(\frac{\partial f}{\partial x}(u,v),\frac{\partial f}{\partial y}(u,v)\right)=(-3u^2-a,2v),$$

which is $(0,0)$ (meaning that (u,v) is singular on E) if and only if $u=(-a/3)^{1/2}$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

and $v=0$. But then $u^3+au+b=v^2$ implies that $4a^3+27b^2=0$, which contradicts our choice of E.

For $F=\mathbb{C}$, any line which intersects E does this in exactly three points, if multiplicities are counted properly. This is a special case of Bézout's theorem (Section 6.4). Let $L=\{(u,v)\in F^2 : v=ru+s\}$ be a line, for some $r,s\in F$. Then

$$L\cap E=\{(u,v)\in F^2 : (ru+s)^2=u^3+au+b \text{ and } v=ru+s\}.$$

Since a,b,r,s are all fixed, this is a cubic equation for u. In the case of a vertical line $L=\{(u,v): v\in F\}$, where $u\in F$ is fixed, one of the points is $\mathcal{O}$.

The group structure. The fundamental property that makes elliptic curves interesting for factorization is that they have a group structure in a natural way. We define the group operation as follows. The negative of a point $P=(u,v)\in E$ is its mirror image $-P=(u,-v)$ upon reflection at the x-axis, and $-\mathcal{O}=\mathcal{O}$. When we intersect the line through P and Q with E, we get three points, say $\{P,Q,S\}$. Then

$$R=P+Q=-S$$

(Figure 19.6 on page 559). Special cases are:

(i) $Q=P$. We take the tangent line at P. Since E is smooth, the tangent is always well defined.

(ii) $Q=\mathcal{O}$. We take the vertical line through P:

$$P+\mathcal{O}=-(-P)=P$$

(iii) $Q=-P$. We take again the vertical line through P and Q and obtain

$$P+(-P)=-\mathcal{O}=\mathcal{O}.$$

It turns out that these definitions make E into a commutative group. The second special case above shows that $\mathcal{O}$ is the neutral element of E, and the third case says that the inverse of a point P is its negative $-P$. As usual, for $k\in\mathbb{Z}$ and $P\in E$ we will write kP for adding P (respectively $-P$ if $k<0$) k times ($-k$ times if $k<0$) to itself, and $0P=\mathcal{O}$.

We now derive the rational expressions for addition on an elliptic curve E. Suppose that $P=(x_1,y_1)$, $Q=(x_2,y_2)$, and $x_1\neq x_2$. Then $R=(x_3,y_3)=P+Q\in E\setminus\{\mathcal{O}\}$. The line through P and Q has the equation $y=\alpha x+\beta$, where $\alpha=(y_2-y_1)/(x_2-x_1)$ and $\beta=y_1-\alpha x_1$. Let $S=(x_3,-y_3)$ be the third intersection point of this line with the curve. Then $(\alpha x_3+\beta)^2={x_3}^3+ax_3+b$. Since x_1,x_2 are

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

the two other roots of the cubic equation $(u^3+au+b)-(\alpha u+\beta)^2=0$, we have $x_1+x_2+x_3=\alpha^2$. It follows that

$$x_3=\left(\frac{y_2-y_1}{x_2-x_1}\right)^2-x_1-x_2,\quad y_3=-y_1+\frac{y_2-y_1}{x_2-x_1}\cdot(x_1-x_3). \tag{9}$$

Thus, the coefficients of the sum of two distinct points are given by rational functions of the input coefficients. We note that these formulas do not explicitly use the Weierstraß coefficients of E, which are determined in fact by the two points on it. A similar formula holds for doubling a point (where $R=2P$, $x_1=x_2$, and $y_1=y_2$; see Exercise 19.15): we have

$$x_3=\left(\frac{3x_1^2+a}{2y_1}\right)^2-2x_1,\quad y_3=-y_1+\frac{3x_1^2+a}{2y_1}\cdot(x_1-x_3), \tag{10}$$

if $y_1\neq 0$, and $2P=\mathcal{O}$ if $y_1=0$.

The curve E with this operation is a commutative group. We have already checked all required properties, except associativity. The latter is not hard to check on a computer algebra system (Exercise 19.17).

The size of an elliptic curve. Our intuition so far has been based on the real numbers. But for the intended application we have to consider elliptic curves over finite fields. Our first task is to determine the size of such an elliptic curve, that is, to estimate the number of points on it. The following estimate is easy and crude.

THEOREM 19.19.
Let E be an elliptic curve over the finite field $\mathbb{F}_q$ of characteristic greater than three. Then $\#E\leq 2q+1$.

PROOF. For each of the q possible values for u, there are at most two possible values for v such that $v^2=u^3+au+b$, corresponding to the two square roots of u^3+au+b. Adding the point at infinity gives the required estimate. □

One reason to think that this is a crude estimate is that, pretending that the value of u^3+au+b varies randomly as u ranges over $\mathbb{F}_q$, we should expect that for about half of the u's there would be two solutions v for the equation, and no solution for the other half. In other words, u^3+au+b should be a square about half of the time. Random elements have this property by Lemma 14.7. More formally, we consider the **quadratic character** $\chi:\mathbb{F}_q^\times\longrightarrow\{1,0,-1\}$ defined by

$$\chi(c)=\begin{cases}1 & \text{if } c \text{ is a square,}\\ 0 & \text{if } c=0,\\ -1 & \text{otherwise.}\end{cases}$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

For q prime, $\chi(c) = (\frac{c}{q})$ is called the Legendre symbol (Section 18.5), and for all $c \in \mathbb{F}_q$,

$$\#\{v \in \mathbb{F}_q : v^2 = c\} = 1 + \chi(c).$$

From this we conclude that

$$\#E = 1 + \sum_{u \in \mathbb{F}_q} (1 + \chi(u^3 + au + b)) = q + 1 + \sum_{u \in \mathbb{F}_q} \chi(u^3 + au + b).$$

If $\chi(u^3 + au + b)$ was a uniformly distributed random variable, then the sum would behave like a random walk on the line. After q steps of such a random walk, we expect to be about $\sqrt{q}$ steps away from the origin (Exercise 19.18). Of course this is not at all a random process, but the analogy provides some intuitive motivation for the following result.

THEOREM 19.20 *Hasse's bound.*
If E is an elliptic curve over the finite field $\mathbb{F}_q$, then $|\#E - (q+1)| \leq 2\sqrt{q}$.

EXAMPLE 19.21. Let $q = 7$. By the Hasse bound, each elliptic curve E over $\mathbb{F}_7$ has $|\#E - 8| \leq 2\sqrt{7}$, so that $3 \leq \#E \leq 13$. Table 19.7 gives the orders of all 42 elliptic curves over $\mathbb{F}_7$.

n	3	4	5	6	7	8	9	10	11	12	13
$\#\{E : \#E = n\}$	1	4	3	6	4	6	4	6	3	4	1

TABLE 19.7: Frequencies of the orders of all elliptic curves E over $\mathbb{F}_7$.

If we take the curve E from Example 19.18 given by $y^2 = x^3 - x$, we can count the points by hand and determine that it contains exactly the eight points

$$(0,0), (1,0), (4,2), (4,5), (5,1), (5,6), (6,0), \mathcal{O}.$$

This group is generated by the two elements $(4,2)$ of order 4 and $(0,0)$ of order 2, and hence is isomorphic to $\mathbb{Z}_4 \times \mathbb{Z}_2$.

Another example is the curve E^* with the equation $y^2 = x^3 + x$, comprising the eight points

$$(0,0), (1,3), (1,4), (3,3), (3,4), (5,2), (5,5), \mathcal{O}.$$

E^* is cyclic and generated, for example, by $(3,3)$. Figure 19.8 illustrates the group structures of E and E^*. $\diamond$

As in any finite group, $(\#E) \cdot P = \mathcal{O}$ for any $P \in E$, by Lagrange's theorem. The order d of P is the smallest positive integer so that $dP = \mathcal{O}$; d is a divisor of $\#E$.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

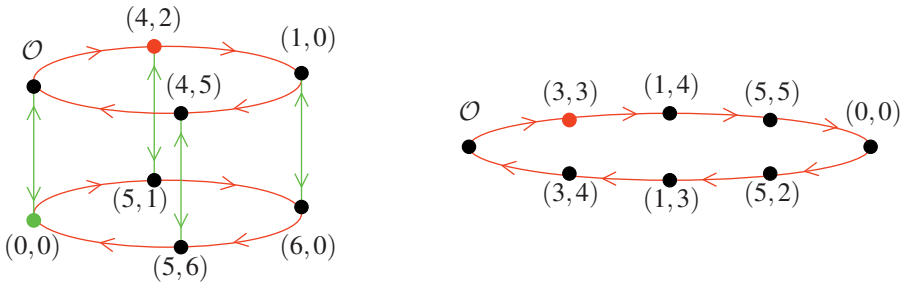


FIGURE 19.8: Structure of the elliptic curve groups E (left) and E^* (right) from Example 19.21. E is generated by $(4,2)$ (red) and $(0,0)$ (green), and E^* is generated by $(3,3)$ (red). There is a colored arrow from a point P to a point Q if $Q-P$ is the generator of that color.

The elliptic curve algorithm. We first state Lenstra's algorithm to factor N, and then prove some of its properties.

ALGORITHM 19.22 Lenstra's elliptic curve factoring method.

Input: $N \in \mathbb{N}$ composite, not a perfect power of an integer, and not divisible by 2 or 3, a bound B on the primes in the base, and a guessed upper bound C for the smallest prime factor of N.

Output: Either a nontrivial divisor of N, or "failure".

1. choose randomly $(a,u,v) \in \{0,\dots,N-1\}^3$
   $b \longleftarrow v^2-u^3-au, \quad g \longleftarrow \gcd(4a^3+27b^2,N)$
   if $1<g<N$ **then return** g **else if** $g=N$ **then return** "failure"

2. let E be the "elliptic curve" over $\mathbb{Z}_N$ with Weierstraß coefficients a,b
   compute all primes $p_1=2<\cdots<p_h$ less or equal to B
   $P \longleftarrow (u,v), \quad Q \longleftarrow P, \quad t \longleftarrow 1$

3. **for** $i=1,\dots,h$ **do**
   $e_i \longleftarrow \lfloor \log_{p_i}(C+2\sqrt{C}+1) \rfloor$
   for $j=0,\dots,e_i-1$ **do**
   { Loop invariants: $t = p_i^j \prod_{1\le r<i} p_r^{e_r}$ and $Q=tP$ }

4. try to compute p_iQ in E, using the formulas (9) and (10) and "repeated doubling"
   if some denominator $w \in \{1,\dots,N-1\}$ which is not invertible modulo N shows up during the computation
   then return $\gcd(w,N)$
   else $Q \longleftarrow p_iQ, \quad t \longleftarrow p_it$

5. **return** "failure"

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

The "elliptic curve" in step 2 is in quotes because the proper definition for a composite N is more complicated. All we need here is that for each prime factor p of N, E mod p is an elliptic curve in the proper sense. In particular, the equations (9) and (10) do *not* make E into a group. This is plausible since some denominator might be nonzero but not invertible, so that the expressions might not be well-defined modulo N. The point is that, until a divisor is found, they give the group structure on the reduction E_p modulo any prime divisor p of N.

We are going to show that successful termination eventually occurs if N has a prime factor below C. (The order of computation, from smaller to larger prime factors, might not be essential for the validity of the algorithm, but is required in the proof given below.)

Let p be a prime divisor of N. Then p does not divide $4a^3+27b^2$, since otherwise the choice is (successfully or unsuccessfully) abandoned in step 1. We denote by E_p the reduction of E modulo p, that is, the elliptic curve over $\mathbb{Z}_p$ with Weierstraß coefficients a,b modulo p. To $P\in E$ corresponds $P_p\in E_p$, just by reducing the coefficients modulo p. Moreover, we let $\mathcal{O}_p$, the point at infinity on E_p, correspond to the point at infinity $\mathcal{O}$ of E. Then $P_p\neq\mathcal{O}_p$ for all $P\in E\setminus\{0\}$, and hence

$$P_p=\mathcal{O}_p \text{ if and only if } P=\mathcal{O}. \tag{11}$$

Then, until the divisor p is found (so that $p\mid\gcd(w,N)$ in step 4), the computation in the algorithm can be considered as implementing arithmetic on E_p in the sense that each partial result $Q=tP$ on E gives, modulo p, the partial result $Q_p=tP_p$ on E_p; in other words, $tP_p=(tP)_p$. The lucky event that provides a factorization occurs when we reach a multiple of the order of P_p on E_p but not of P_q on E_q, for two prime divisors p,q of N. We had a similar situation in Pollard's $p-1$ method, with the elliptic curve replaced by the group of units.

LEMMA 19.23. *Suppose that (E,P) is chosen, p,q are distinct prime divisors of N, l is the largest prime factor of the order of P_p in the group E_p, $p\le C$, $\#E_p$ is B-smooth, and $l\nmid\#E_q$. Then the algorithm factors N.*

PROOF. We let $k=\prod_{1\le r\le h}p_r^{e_r}$, with e_r as in step 3 for $1\le r\le h$. The loop invariants are easily checked by induction on i and j. Since $\#E_p$ is B-smooth and $p\le C$, the Hasse bound implies that $\#E_p\mid k$. Let d be the order of P_p in E_p. Then $d\mid\#E_p$, and hence $l\le B$ and $d\mid k$. Let $p_i=l$ and e be the exponent of l in d, so that $1\le e\le e_i$. When $j=e-1$, then

$$t=l^{e-1}\prod_{1\le r<i}p_r^{e_r} \text{ and } Q=tP$$

holds before step 4, $t\not\equiv 0 \bmod d$, and $lt\equiv 0 \bmod d$. It follows that $Q_p=tP_p\neq\mathcal{O}_p$, and $lQ_p=ltP_p=\mathcal{O}_p$. Therefore, if the algorithm ever succeeded in computing lQ,

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

the result could only be $\mathcal{O}$. We show that in fact it terminates by finding a divisor before reaching this situation.

Assume to the contrary that $lQ = \mathcal{O}$ is computed in step 4 by the algorithm. Since this computation also implements arithmetic on E_q, we have computed $lQ_q = (l\,tP)_q = \mathcal{O}_q$. But then, since l does not divide $\#E_q$ or the order of P_q, the point $Q_q = tP_q$ is already $\mathcal{O}_q$. It then follows that $Q = \mathcal{O}$ by (11) and hence $Q_p = \mathcal{O}_p$, a contradiction. □

We now turn to the analysis of the probability that the assumptions in this lemma are satisfied. We note that $\text{prob}(l \nmid \#E_q)$ is almost 1 (see Notes 19.7), so we only consider the probability of the randomly chosen parameters to produce an elliptic curve E_p with $\#E_p$ being B-smooth. This is basically settled by the following result of Lenstra's, whose proof is outside the scope of this text.

THEOREM 19.24.
There exists a $c \in \mathbb{R}_{>0}$ with the following property. Let p be prime, $S \subseteq \mathbb{N}$ with $S \subseteq (p+1-\sqrt{p},\, p+1+\sqrt{p})$ and $\#S \geq 3$, and $a, b \in \mathbb{F}_p$ chosen at random. Let

$$E_p = \{(u,v) : v^2 = u^3 + au + b\} \cup \{\mathcal{O}\}$$

be an elliptic curve over $\mathbb{F}_p$. Then

$$\text{prob}\{\#E_p \in S\} \geq \frac{c \cdot \#S}{\sqrt{p}\log p}.$$

We note that S is taken from the middle half of the range given by the Hasse bound; we have $S \subseteq \{6,7,8,9,10\}$ for $p = 7$. Thus the sizes of elliptic curves are roughly equally distributed in this middle half. Taking the set of B-smooth numbers for S and using Lemma 19.23, we have the following consequence.

COROLLARY 19.25.
There exists a $c \in \mathbb{R}_{>0}$ with the following property. Let $p \leq C$ be a prime divisor of N, and

$$\begin{aligned}\sigma &= \#\{B\text{-smooth numbers in } (p+1-\sqrt{p}, p+1+\sqrt{p})\} \\ &= \psi(p+1+\sqrt{p}, B) - \psi(p+1-\sqrt{p}, B),\end{aligned}$$

where ψ is the "number of smooth numbers" function defined in (4). If $\sigma \geq 3$, then the number M of triples $(a,u,v) \in \{0,\ldots,N-1\}^3$ for which the algorithm factors N satisfies

$$\frac{M}{N^3} \geq \frac{c\sigma}{\sqrt{p}\log p}.$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

How many trials of the algorithm are necessary for successful factoring with high probability? We let $s = \sigma/(2\sqrt{p})$ denote the probability for a random number in the range $(p+1-\sqrt{p}, p+1+\sqrt{p})$ to be B-smooth. If we run the algorithm repeatedly, say m times, then the failure probability is at most

$$\left(1-\frac{M}{N^3}\right)^m \le \left(1-\frac{sc}{\ln p}\right)^m \le \left(1-\frac{sc}{\ln C}\right)^m \le e^{-msc/\ln C} \le \varepsilon,$$

when we choose $m \ge \ln(1/\varepsilon)\ln(C)/(sc)$, where c is as in the previous corollary.

One addition or doubling according to (9) and (10) takes a constant number of arithmetic operations modulo N, and the time for one execution of step 4 is $O(\log p_i)$ operations modulo N. (It is amusing to note that the formula for addition is cheaper in this representation than the formula for doubling, requiring only two vs. three multiplications modulo N, plus one division and some additions.) Thus the overall cost of the algorithm is $O(\sum_{i\le h} e_i \log p_i)$ or $O(B\log C)$ operations modulo N, and the number of arithmetic operations modulo N for m executions is

$$O(m\cdot B\log C) \text{ or } O(m\cdot B\log N). \tag{12}$$

Increasing B makes σ and s larger, hence m smaller, so that there is a trade-off in this time estimate. To choose B optimally, we would like to use a smoothness result like Theorem 19.13, but for random numbers from a small interval. Unfortunately, no proof of the following is known.

CONJECTURE 19.26. *For positive real numbers x, u, and an integer d chosen uniformly at random from the interval $(x-\sqrt{x}, x+\sqrt{x})$, we have*

$$\text{prob}\,\{d \text{ is } x^{1/u}\text{-smooth}\} = u^{-u(1+o(1))}.$$

For a rough estimate of the cost, we take $u = \ln p/\ln B$, so that $B = p^{1/u}$. Ignoring the term $o(1)$, the conjecture says that s is about u^{-u} and m about $\ln C\cdot u^u$. We want to minimize the factor mB in (12). Its logarithm is about

$$\ln(u^u) + \ln B = \frac{\ln p}{\ln B}\cdot\ln\left(\frac{\ln p}{\ln B}\right) + \ln B. \tag{13}$$

Setting

$$B = e^{\sqrt{(\ln p\cdot\ln\ln p)/2}} = L(p)^{1/\sqrt{2}}, \tag{14}$$

with the function L defined in (8), we have

$$\frac{\ln p}{\ln B} = \frac{\sqrt{2}\ln p}{(\ln p\cdot\ln\ln p)^{1/2}} = \left(\frac{2\ln p}{\ln\ln p}\right)^{1/2}.$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

The right hand side in (13) is, when $\operatorname{lnln} p \geq 2$, at most

$$\left(\frac{2\ln p}{\operatorname{lnln} p}\right)^{1/2} \cdot \frac{1}{2}\operatorname{lnln} p + \ln B = \sqrt{2}(\ln p \cdot \operatorname{lnln} p)^{1/2}.$$

Thus the number of word operations is, assuming the conjecture,

$$e^{(\sqrt{2}+o(1))\sqrt{\ln p \cdot \operatorname{lnln} p}} = L(p)^{\sqrt{2}+o(1)}.$$

We note that $L(p)^{\sqrt{2}} \leq L(N)$ for any $p \leq N^{1/2}$. In an implementation, we do not know p and have to guess good values for B and C; the algorithm works well if there is a prime factor $p \leq C$ for which (14) holds approximately. One usually guesses a "small" initial value of C, determines B according to (14) with p replaced by C, and doubles the value of C in case the algorithm does not produce a factor. In the complete factorization of N, the second largest prime factor p will usually require the major portion of the effort. The particular strength of the elliptic curve method is that it profits from the presence of prime factors that are much smaller than $N^{1/2}$; Section 19.1 tells some of the success stories.

Notes. **19.1.** Table 19.2 is from Brent (1999).

19.2. The estimates for random integers are from Knuth & Trabb Pardo (1976).

19.3. Algorithm 19.2 is from Strassen (1976). An algorithm using a similar method was earlier given by Pollard (1974). It yields the same asymptotic result, but is quite a bit more complicated.

19.4. The annoying assumption in Pollard's ρ method that x^2+1 is random was an open problem for a long time, until Horwitz & Venkatesan (2002) found a variant for which they could prove the required randomness. Brent (1980) gives an improvement of Pollard's method. Brent & Pollard (1981) used it to factor F_8. Knuth (1998) attributes the cycle detection method to Floyd.

19.5. The simple observation that $s^2 - t^2 = (s+t)(s-t)$ leads to a description of all **Pythagorean triples** $(x,y,z) \in \mathbb{N}^3$ with $x^2+y^2=z^2$; see Exercise 19.8. A method for factoring integers based on congruent squares is designed in Kraïtchik (1926), Chapter 16. Theorem 19.13 is from Canfield, Erdős & Pomerance (1983). The approach via Lemma 19.14 (and its proof) are in Schnorr (1982). Charlie Rackoff has proposed a variant (Exercise 19.10). Pomerance (1982) gives an improvement of Dixon's method using $O^{\sim}(L(N)^{\sqrt{5/2}})$ word operations.

An important variant of the quadratic sieve, the **multiple polynomial quadratic sieve**, is practically useful for distributed computation on a network of workstations (Caron & Silverman 1988). See Silverman (1987) and Pomerance (1990) for an overview.

19.6. The $p-1$ method is from Pollard (1974).

19.7. Elliptic curves and the role of the "point at infinity" are best understood in the framework of projective geometry. The **projective plane** $\mathbb{P}^2$ over F consists of all triples $(u:v:w)$ with $u,v,w \in F^3$, not all zero, where we identify two such triples if they are multiples of each other. We may also regard $(u:v:w)$ as the line in F^3 through (u,v,w) and the origin. The projective curve in $\mathbb{P}^2$ corresponding to an elliptic curve E given by $y^2 = x^3+ax+b$ is

$$\tilde{E} = \{(U:V:W) \in \mathbb{P}^2 : V^2W = U^3 + aUW^2 + bW^3\},$$

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

and $E \cap F^2$ is in correspondence with the **affine part** $\tilde{E} \cap \{W \neq 0\} = \tilde{E} \cap \{W = 1\}$ via the substitution $u = U/W, v = V/W$. The intersection with the **line at infinity** is

$$\tilde{E} \cap \{W = 0\} = \{(U : V : W) \in \mathbb{P}^2 : W = U = 0\} = \{(0 : 1 : 0)\} = \{\mathcal{O}\}.$$

The choice of 1 for the second coordinate is arbitrary.

In order to get a better feeling for the required algebraic geometry, the reader is advised to consult Lenstra's (1987) article and one of the texts in algebraic geometry, such as Chapters 1 and 4 of Hartshorne (1977), Fulton (1969), Brieskorn & Knörrer (1986), Section VI.1 of Koblitz (1987a), or the detailed books by Silverman (1986) and Cox (1989).

Over the complex numbers, an elliptic curve is a one-dimensional curve and hence a two-dimensional real surface that looks like a torus. You can think of the real picture as being the intersection of such a torus with a plane (in four dimensions).

In general, one invariant of complex curves is the **genus** g. In the real surface representation, g is the number of *holes*. If $E \subseteq \mathbb{C}^2$ is a nonsingular planar curve defined by an irreducible polynomial of degree d (and also nonsingular at infinity), then $g = (d-1)(d-2)/2$. For an elliptic curve, we have $d = 3$ and $g = 1$. Also, $g = 0$ if and only if the curve is isomorphic to the projective line $\mathbb{P}^1$, which is defined analogously to the projective plane. (An example is the parabola given by $y = x^2$.)

No curve of genus ≥ 2 is a group with a rational operation. Of course any curve (and any set) can be made into a group, since a group of the same cardinality exists, and one can translate the group operations via a bijection. However, the above says that if we want to express the group operation by rational functions in the coordinates, as done in Section 19.7 for elliptic curves, then this is impossible for genus ≥ 2.

Manin (1956) and Chahal (1995) present proofs of Hasse's (1933) bound 19.20 that use only "elementary" methods. This bound had been conjectured by Emil Artin in 1923. It is a special case of the famous **Weil bound**, which says that $|\#X - (q+1)| \leq 2g\sqrt{q}$ for a nonsingular projective algebraic curve X over $\mathbb{F}_q$ of genus g. A variant of Weil's bound, also valid for singular curves, is given in Bach (1996).

An elliptic curve group over a finite field is either cyclic or a direct product of two cyclic groups, as illustrated in Figure 19.8 (see Silverman 1986).

A fixed prime number ℓ divides a random number with probability about $1/\ell$. In view of Theorem 19.24, one might expect that ℓ divides $\#E$, where E is a random elliptic curve over a finite prime field $\mathbb{F}_p$, with probability about $1/\ell$ as well. This is false, however. Lenstra (1987) shows that the probability tends to $1/(\ell-1)$ and $\ell/(\ell^2-1)$ as p tends to infinity over the primes with $p \not\equiv 1 \bmod \ell$ and $p \equiv 1 \bmod \ell$, respectively.

Further notes. We list some other integer factorization algorithms. The $\boldsymbol{p+1}$ **method** by Guy (1975) and Williams (1982) and the $\boldsymbol{\Phi_k(p)}$ **method** by Bach & Shallit (1988) are generalizations of Pollard's (1975) $p-1$ method. ($\Phi_k \in \mathbb{Z}[x]$ is the kth cyclotomic polynomial; see Section 14.10). These methods work well if N has a prime factor with the property that $\Phi_k(p)$ is smooth.

The **continued fraction method** by Morrison & Brillhart (1971, 1975) is a different approach to generating B-numbers, where the continued fraction expansion (Section 4.6) of $\sqrt{N}$ is used to get the b's. By a heuristic argument, their algorithm should use $L(N)^{O(1)}$ steps, thus predating Dixon's result, but this was not proven rigorously. (They factored F_7 and introduced several other important ideas, such as using a factor base and linear algebra

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

modulo 2 to find squares.) Already Lehmer & Powers (1931) had used this expansion to find two squares that are congruent modulo N. Pomerance (1982) exhibits variants that use $L(N)^{\sqrt{3/2}+o(1)}$ word operations, under some unproven hypotheses. Further discussions are in Pomerance & Wagstaff (1983) and Williams & Wunderlich (1987). The origins of this method can already be found in Legendre (1785), § XV. (In the Berkeley library copy that we consulted, D. H. Lehmer has corrected a calculation error of Legendre's.)

The **number field sieve** by Lenstra, Lenstra, Manasse & Pollard (1990) runs in time $\exp(O(\sqrt[3]{\log N(\log\log N)^2}))$. It was the first general asymptotic progress (in terms of the order of the exponent) since Dixon's (1981) random squares method. Lenstra & Lenstra (1993) give a status report. The original approach was designed for numbers of a special form (as they occur in the Cunningham project), but newer versions apply to arbitrary numbers; see Dodson & Lenstra (1995) and Cowie, Dodson, Elkenbracht-Huizing, Lenstra, Montgomery & Zayer (1996) about their efficiency. In 1999, it was used to break the 211-digit repunit $(10^{211}-1)/9$ into its two prime factors, with 93 and 118 digits.

As we have seen, the analyses of several factoring algorithms rely on unproven conjectures. The current world records on *rigorously* proven upper bounds on integer factoring algorithms are Pollard's and Strassen's $O^{\sim}(N^{1/4})$ for deterministic methods and $L(N)^{1+o(1)}$, due to Lenstra & Pomerance (1992), for probabilistic algorithms.

Exercises.

19.1$^{\longrightarrow}$ Prove that the quotient N of $2^{599}-1$ divided by its 23-digit prime factor

$$16\,659\,379\,034\,607\,403\,556\,537 \tag{15}$$

is composite. N has 159 decimal digits.

19.2* (Lenstra 1990) Consider the following special polynomial factorization task: input is a prime p and $f \in \mathbb{F}_p[x]$ of degree n and dividing $x^p - x$, so that all monic irreducible factors of f in $\mathbb{F}_p[x]$ are linear and distinct. Adapt the Pollard and Strassen method to find a deterministic algorithm for factoring f with $O^{\sim}(n\sqrt{p})$ operations in $\mathbb{F}_p$ if $p^2 > n$.

19.3 Factor the integer $N = 23\,802\,996\,783\,967$ using Pollard's ρ method, and also with the Pollard and Strassen method.

19.4 Let p be a prime. For a sequence $u = (u_i)_{i\in\mathbb{N}} \in \mathbb{Z}_p^{\mathbb{N}}$ let $S(u) = \min\{i \in \mathbb{N}: \exists j < i \;\; u_j = u_i\}$ be the least index with a collision.

(i) For any $u_0 \in \mathbb{Z}_p$, we define a sequence $u = (u_i)_{i\in\mathbb{N}} \in \mathbb{Z}_p^{\mathbb{N}}$ by $u_i = u_{i-1}^2 + 1$ if $i \geq 1$, as in Pollard's algorithm 19.8. Determine the mean value (over the choices of u_0) of $S(u)$ for $p = 167$ and $p = 179$ by trying all possible initial values of u_0. Compare your result with the estimated expected value of $S(u)$ for random sequences from the proof of Theorem 19.5.

(ii) Determine the mean value of $T(u) = \min\{i \in \mathbb{N}_{>0}: u_i = u_{2i}\}$, with u and p as in (i), for all possible values of u_0. Compare to your results of (i).

19.5 (Guy 1975) Let $x_0 = 2$ and $x_i = x_{i-1}^2 + 1$ for $i \geq 1$. For $p \in \mathbb{N}$, we let $e(p) = \min\{i \in \mathbb{N}_{\geq 1}: x_i \equiv x_{2i} \bmod p\}$.

(i) Calculate $e(p)$ for the primes $p \leq 11$.

(ii) Calculate $e(p)$ for the primes $p \leq 10^6$. You should find $e(p) \leq 3680$ for all these p. (Guy (1975) notes that $e(p)$ seems to grow like $(p \ln p)^{1/2}$.)

(iii) Let N be a number to be factored, run Pollard's ρ method on it with initial value $x_0 = 2$, and assume that $\gcd(x_i - x_{2i}, N) = 1$ for $1 \leq i \leq k$. Show that $e(p) > k$ for all prime divisors p of N.

(iv) Conclude that if the gcd in (iii) is trivial for 3680 steps, then N has no factor up to 10^6.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

19.6 Prove that

$$\left(\int_{-\infty}^{\infty} e^{-x^2} dx\right)^2 = \pi. \tag{16}$$

Hint: Write (16) as a two-dimensional integral, and use the substitution $(x,y) = (r\cos\varphi, r\sin\varphi)$.

19.7$^{\longrightarrow}$ Factor the number N from Exercise 19.3 using Dixon's random squares method with all primes less than $B = 40$ as your factor base.

19.8 Three positive integers $(x,y,z) \in \mathbb{N}^3$ form a **Pythagorean triple** if $x^2+y^2 = z^2$, and a **primitive Pythagorean triple** if furthermore $\gcd(x,y,z) = 1$.

(i) Show that any Pythagorean triple is of the form $(\lambda x, \lambda y, \lambda z)$ for a primitive Pythagorean triple (x,y,z) and some $\lambda \in \mathbb{N}$.

(ii) Let $s,t \in \mathbb{N}$ be coprime with $s > t$ and st even. Show that $(s^2-t^2, 2st, s^2+t^2)$ is a primitive Pythagorean triple.

(iii) Let (x,y,z) be a primitive Pythagorean triple. Show that z is odd and either x or y is odd, but not both. (Hint: Calculate modulo 4.) Assume that x is odd. Prove that $(z+x)/2$ and $(z-x)/2$ are coprime squares, not both odd, and conclude that (x,y,z) is of the form as in (ii).

(iv) Use (ii) and (iii) to generate all primitive Pythagorean triples (x,y,z) with $z \le 100$.

19.9 What is the relation between $n^{1+o(1)}$ and $O^{\sim}(n)$?

19.10** In Lemma 19.14, we proved that the numbers $a^2 \text{ rem } N$ for random a are smooth sufficiently often. We can avoid the use of this lemma by the following modifications to Dixon's random squares algorithm 19.12, as proposed by Charlie Rackoff.

- In step 1: $u \longleftarrow (\ln N)/\ln p_h$, $\quad \tau \longleftarrow \lambda u^u$ with a small constant λ, say $\lambda = 10$
  choose $y \in \{1,\dots,N-1\}$ at random, $\quad g \longleftarrow \gcd(y,N)$, $\quad$ **if** $1 < g < N$ **then return** g
- In step 4: $a \longleftarrow b^2 y \text{ rem } N$
- In step 7: **until** $\#A = \tau$
- In step 8: l must be even
- In step 9: $s \longleftarrow y^{l/2} \prod_{1 \le i \le l} b_i$

We denote by $\sigma = \psi(N,p_h)/\varphi(N)$ the probability for $x \in \{1,\dots,N-1\}$ with $\gcd(x,N) = 1$ to be B-smooth. Thus σ is about τ^{-1}. We let $N = q_1^{l_1} \cdots q_t^{l_t}$ be the prime factorization of N, with pairwise different primes $q_1,\dots,q_t$. For each $i \le t$, there are exactly $(q_i-1)q_i^{l_i-1}/2$ squares modulo $q_i^{l_i}$, and the same number of nonsquares. In a somewhat audacious notation we write $+S_i \subset \mathbb{Z}^{\times}_{q^{l_i}}$ for the set of squares, and $-S_i \subseteq \mathbb{Z}^{\times}_{q^{l_i}}$ for the nonsquares.

Then the Chinese Remainder Theorem gives a decomposition

$$\mathbb{Z}_N^{\times} \cong \bigcup_{\varepsilon_1,\dots,\varepsilon_t \in \pm} \varepsilon_1 S_1 \times \varepsilon_2 S_2 \times \cdots \times \varepsilon_t S_t = \bigcup_{\varepsilon \in \pm^t} T_\varepsilon$$

of $\mathbb{Z}_N^{\times}$ into 2^t subsets T_ε of equal size.

If $y \in T_\varepsilon$, then also all a computed in step 4 are in T_ε. If T_ε has its fair share of B-smooth numbers, namely about $\sigma \cdot \#T_\varepsilon$ many, then the algorithm will work well with that choice of y. However, we do not know that the smooth numbers are equally distributed over the 2^t sets T_ε. So the first question is to show that a reasonable fraction of all y's is sufficiently good.

(i) Let $A = \bigcup_{i \in I} B_i$ be a partition of a finite set A into disjoint subsets of equal size $k = \#A/\#I$, $C \subseteq A$, and $s = \#C/\#A$. Then for at least $s \cdot \#I/2$ indices $i \in I$ we have $\#(B_i \cap C) \ge sk/2$.

(ii) Show that for a fraction at least $\sigma/2$ of the $\varepsilon \in \pm^t$, T_ε contains a fraction at least $\sigma/2$ of B-smooth numbers. Hint: Apply (i) to $A = \mathbb{Z}_N^{\times}$, C the B-smooth numbers, so that $s = \sigma$, and the partition into the subsets T_ε.

(iii) Analyze the success probability and the running time of the algorithm described above.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

19.11 Check that the curve $E = \{(x,y) \in \mathbb{F}_7^2 : y^2 = x^3 + x + 3\}$ over $\mathbb{F}_7$ is nonsingular. Compute all points on it, and verify that it is cyclic and generated by $(4,1)$.

19.13 Let E be an elliptic curve and $P, Q \in E$. Explain why $P + Q = S$, where S is the third intersection point of E with the line through P and Q (see Figure 19.6), is not a group operation.

19.14 Let F be a field and $f = x^3 + ax + b \in F[x]$.

(i) Check that $r = \operatorname{res}(f, f') = 4a^3 + 27b^2$.

(ii) Conclude that f is squarefree if and only if $r \neq 0$.

(iii) For which values of b does $y^2 = x^3 - x + b$ not define an elliptic curve over $F = \mathbb{R}$? Plot the curves for all these values.

19.15 Let $E = \{(x,y) \in F^2 : y^2 = x^3 + ax + b\}$ be an elliptic curve over a field F and $P = (x_1, y_1) \in E$. Determine the equation of the tangent to E through P (distinguish the two cases $y_1 = 0$ and $y_1 \neq 0$), and prove that the doubling formula (10) realizes the geometric description using the tangent line.

19.16 Show that an elliptic curve E has at most three points P of order 2, for which $P \neq \mathcal{O}$ and $2P = \mathcal{O}$.

19.17$^{\longrightarrow}$ You are to check associativity of the addition we defined on an elliptic curve E.

(i) Write a procedure add to calculate the sum of two distinct points, using (9).

(ii) Check that for three points P, Q, R,

$$\text{ass} = \text{add}(\text{add}(P,Q),R) - \text{add}(P,\text{add}(Q,R))$$

is not zero.

(iii) What has gone wrong in (ii)? We have not used that the three points lie on the same curve. Calculate the Weierstraß coefficients a, b from P and Q, set $f = y_3^2 - (x_3^3 + ax_3 + b)$, where $R = (x_3, y_3)$, and check that ass $\equiv 0 \bmod f$. (You may have to simplify and take numerators at the appropriate place.)

(iv) We now have associativity at three "generic" points P, Q, R. Check associativity when one of them is $\mathcal{O}$.

(v) It remains to check the cases where two points coincide, say $P = Q$ or $P + Q = R$, so that (9) is not applicable. You have two ways of doing this: writing a little program for these cases, or arguing by continuity. The latter requires some algebraic geometry.

19.18** (i) Prove that $\sum_{0 \le k < n} \binom{2n}{k} = (4^n - \binom{2n}{n})/2$ and $\sum_{0 \le k < n} \binom{2n-1}{k} = 4^{n-1}$, for all positive integers n.

(ii) Let $n \in \mathbb{N}_{>0}$, X_i for $1 \le i \le 2n$ be a collection of independent random variables which take on each of the two values 1 and -1 with probability $1/2$, and $X = \sum_{1 \le i \le 2n} X_i$ be a random walk of length $2n$. Prove that $\text{prob}(X = 2(n-k)) = \text{prob}(X = -2(n-k)) = \binom{2n}{k} 4^{-n}$ for $0 \le k \le n$.

(iii) Show that $\mathcal{E}(X) = 0$ and $\mathcal{E}(|X|) = 2n \binom{2n}{n} 4^{-n}$.

(iv) Use Stirling's formula $n! \in \sqrt{2\pi n}(n/e)^n(1 + O(n^{-1}))$ (see Graham, Knuth & Patashnik 1994) to show that $\mathcal{E}(|X|) \in 2\pi^{-1/2} n^{1/2} + O(n^{-1/2})$.

(v) Prove the same formulas as in (iii) when there are $2n-1$ instead of $2n$ random variables.

19.19$^{\longrightarrow}$ Program Lenstra's algorithm 19.22, and use it to factor the number N from Exercise 19.3 with $B = 40$ and $C = 12\,000$.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press

Real mathematics has no effects on war. No one has yet discovered any warlike purpose to be served by the theory of numbers or relativity; and it seems very unlikely that anyone will do so for many years.

Godfrey Harold Hardy (1940)

Die reine Zahlentheorie ist dasjenige Gebiet der Mathematik, das bisher noch nie Anwendung gefunden hat.[1]

David Hilbert (1930)

It would not be an exaggeration to state that *abstract* cryptography is *identical* with abstract mathematics.

Abraham Adrian Albert (1941)

Give me problems, give me work, give me the most abstruse cryptogram, or the most intricate analysis, and I am in my own proper atmosphere.

Sir Arthur Conan Doyle (1890)

"Right. So I have a translation key and you have a signature key and all the communication from you to me needs both those keys to encode and decode it properly. But if I want to send a message back, I can't use those same keys—I need *my* signature key and *your* translation key."
"And Joe has a different translation key and when I send him a message I have to use *his* key. And that's how everybody is approaching this, and doing it that way has the kinds of problems we're sitting here to solve."

Philip Friedman (1996)

The KGB, more than other foreign-intelligence agencies, still depended on one-time-pad cipher systems. These were unbreakable, even in a theoretical sense, unless the code sequence itself were compromised.

Tom Clancy (1988)

[1] Pure number theory is that part of mathematics for which up to now no application has ever been found.

https://doi.org/10.1017/CBO9781139856065.024 Published online by Cambridge University Press